

Tresaurio — Teknik Altyapı Dokümantasyonu

Kapsamlı kripto trading terminali ve portföy yönetim platformu.

Bu doküman projenin tüm teknik katmanlarını — frontend, backend, veritabanı, state management, servisler ve deployment — eksiksiz açıklar.

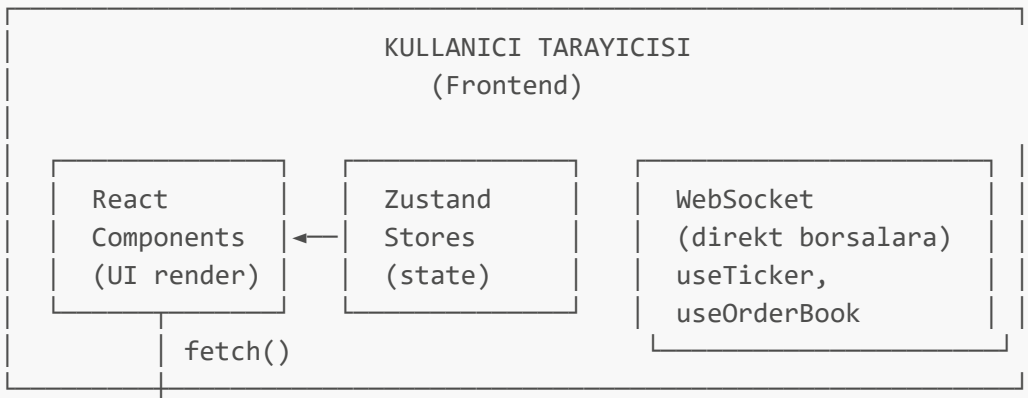
İçindekiler

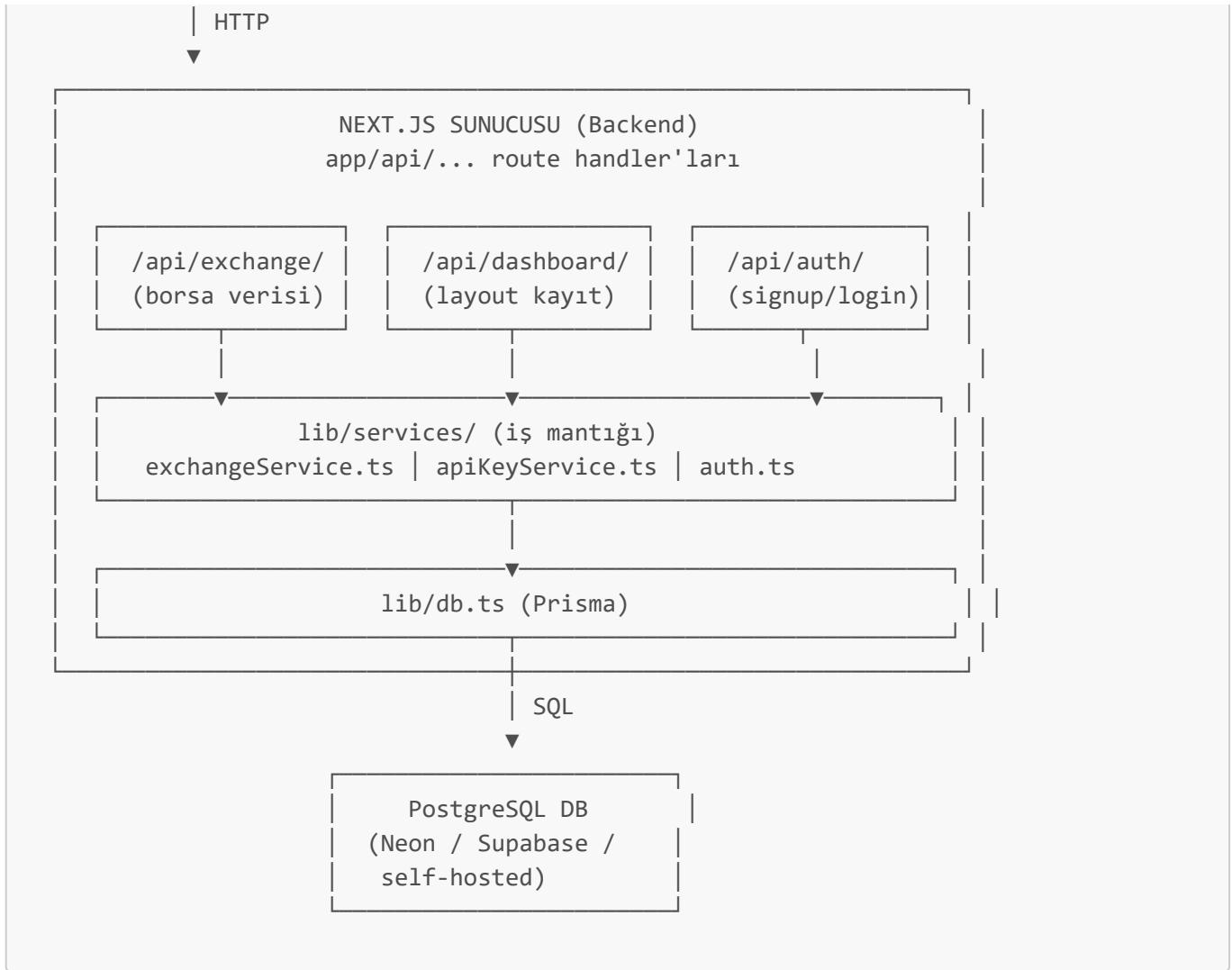
0. Sistem Mimarisi — Frontend/Backend Ayrımı
1. Projeye Genel Bakış
2. Dizin Yapısı
3. Frontend Mimarisi
4. State Management — Zustand
5. Backend — API Routes
6. Veritabanı — Prisma & PostgreSQL
7. Servisler & Kütüphaneler
8. Authentication Sistemi
9. Dış Entegrasyonlar
10. Custom Hooks
11. Personalized Dashboard Modül Sistemi
12. TypeScript Tipleri
13. Environment Variables
14. Deployment & Altyapı
15. Güvenlik
16. Performans Optimizasyonları
17. Geliştirici Rehberi

0. Sistem Mimarisi — Frontend/Backend Ayrımı

Bu bölüm, projenin genel yapısını ve hangi kodun nerede çalıştığını özetler. Kalanı okumadan önce bu bölümü anlamak yeterlidir.

0.1 Nerede Ne Çalışır?





Ayrıca bazı bileşenler borsalara doğrudan WebSocket bağlantısı kurar (tarayıcıdan, Next.js'i atlatarak):

```

Tarayıcı → wss://stream.binance.com (useTicker, useOrderBook)
Tarayıcı → wss://ws.okx.com (useTicker, useOrderBook)
Tarayıcı → https://min-api.cryptocompare.com (useNews)

```

0.2 Frontend Nedir, Ne Yapar?

Çalıştığı yer: Kullanıcının tarayıcısı

Dizinler: `components/`, `hooks/`, `store/`, `app/(sayfalar)/`

Katman	Dosyalar	Görevi
Sayfalar	<code>app/**/page.tsx</code>	Route'a karşılık gelen ekran
Bileşenler	<code>components/**/* .tsx</code>	UI parçaları (button, form, modül)
Hooks	<code>hooks/* .ts</code>	Veri çekme + WebSocket bağlantısı
Store	<code>store/* .ts</code>	Global client-side state (Zustand)

Frontend hiçbir zaman:

- Prisma'ya doğrudan erişmez
- Şifresi çözülmüş API key'leri görmez
- Veritabanına direkt sorgu atmaz

0.3 Backend Nedir, Ne Yapar?

Çalıştığı yer: Next.js sunucusu (Vercel function veya Node.js process)

Dizinler: `app/api/`, `lib/services/`, `lib/db.ts`, `lib/encryption.ts`

Katman	Dosyalar	Görevi
API Routes	<code>app/api/**/*.route.ts</code>	HTTP endpoint'leri — frontend'den gelen istekleri karşılar
Servisler	<code>lib/services/*.ts</code>	İş mantığı — borsa API'leri, şifreleme, key yönetimi
DB katmanı	<code>lib/db.ts</code> + Prisma	Veritabanı sorguları
Auth	<code>lib/auth.ts</code>	Session doğrulama

Backend her zaman:

- API key'leri şifreli saklar ve sadece imzalı istek için çözer
- Session'ı doğrular (`await auth()`) korunan endpoint'lerde
- Borsalara yapılan authenticated istekleri proxy'ler (client değil, sunucu yapar)

0.4 Veri Akışı — Örnek Senaryolar

Senaryo 1: Kullanıcı futures pozisyonlarını görüntüler

1. FuturesPositionsModule (React bileşeni) render olur
2. useEffect → fetch("/api/exchange/positions?exchange=binance")
3. API route → auth() ile session doğrular
4. → apiKeyService.getDecryptedApiKey(userId, "binance")
5. → Prisma → PostgreSQL → şifreli key çekilir
6. → encryption.decrypt(key)
7. → exchangeService.getFuturesPositions(decryptedKey)
8. → HMAC imzalı istek → api.binance.com
9. Yanıt → JSON → fetch yanıtı → useState → UI render

Senaryo 2: Dashboard kaydedilir

1. Kullanıcı modülü taşır
2. personalizedDashboardStore → modules state güncellenir
3. localStorage'a anında yazılır (offline yedek)
4. 3 saniye debounce dolunca → fetch("POST /api/dashboard")
5. API route → Prisma → userDashboard.upsert()
6. PostgreSQL'e JSON layout kaydedilir

Senaryo 3: Canlı fiyat görüntülenir

1. useTicker hook → WebSocket bağlantısı açar
wss://stream.binance.com:9443/ws/btcusdt@ticker
2. Mesaj gelir → priceStore.updatePrice() çağrılır
3. Zustand store güncellenir
4. Tüm abone bileşenler (live-prices modülü vb.) re-render olur
5. (WebSocket düşerse → REST fallback → /api/v2/binance/ticker)

Senaryo 4: Kullanıcı kayıt olur

1. SignupForm → fetch("POST /api/auth/signup")
2. API route → email unique mi kontrol et (Prisma)
3. → bcrypt.hash(password, 12)
4. → prisma.user.create()
5. → emailVerified anında set edilir
6. → NextAuth oturumu açılır

0.5 Hangi Kod Nerede Çalışır — Hızlı Referans

Dosya / Klasör	Nerede çalışır	Ne yapar
app/**/page.tsx	Sunucu (SSR) + tarayıcı	Sayfalar
app/api/**/route.ts	Sadece sunucu	API endpoint'leri
lib/db.ts	Sadece sunucu	Prisma bağlantısı
lib/services/	Sadece sunucu	İş mantığı
lib/encryption.ts	Sadece sunucu	AES şifreleme
lib/auth.ts	Sadece sunucu	NextAuth config
components/**	Sadece tarayıcı	UI render
hooks/**	Sadece tarayıcı	Veri çekme, WebSocket
store/**	Sadece tarayıcı	Global state
services/WebSocketService.ts	Sadece tarayıcı	WS bağlantı yönetimi

1. Projeye Genel Bakış

Alan	Değer
İsim	Tresaurio
Tip	Kripto Trading Terminali & Portföy Yönetim Platformu

Alan	Değer
Framework	Next.js 15 (App Router)
Dil	TypeScript 5
Veritabanı	PostgreSQL (Prisma ORM)
State	Zustand v5
Auth	NextAuth.js v5
Styling	Tailwind CSS v4

Ne yapar?

- Birden fazla kripto borsasına (Binance, OKX, Bybit, Coinbase, Hyperliquid) bağlanır
- Gerçek zamanlı piyasa verisi sunar (WebSocket + REST fallback)
- Portföy ve P&L takibi yapar
- Kullanıcıya özel, modüler bir dashboard canvas'ı sunar (50+ widget)
- Şifreli API key yönetimi sağlar
- Whale takibi, haber akışı, multi-chain cüzdan sorgulama gibi araçlar içerir

2. Dizin Yapısı

```

tresaurio/
├── app/                                # Next.js App Router
│   ├── (workspace)/                  # Kimliği doğrulanmış görünüm (layout grubu)
│   │   ├── layout.tsx
│   │   └── personalized-dashboard/
│   │       └── page.tsx
│   ├── admin/                       # Admin paneli (şifreyle korunan)
│   │   ├── dashboard/page.tsx
│   │   ├── login/page.tsx
│   │   └── page.tsx
│   └── api/                          # RESTful API route'ları
│       ├── admin/users/route.ts
│       ├── analytics/pnl/route.ts
│       ├── auth/
│       │   ├── [...nextauth]/route.ts
│       │   ├── forgot-password/route.ts
│       │   ├── reset-password/route.ts
│       │   ├── signup/route.ts
│       │   └── verify-email/route.ts
│       ├── dashboard/
│       │   ├── favorites/route.ts
│       │   ├── route.ts
│       │   └── templates/
│       │       ├── [id]/route.ts
│       │       └── route.ts

```

```

├── exchange/
│   ├── account/route.ts
│   ├── avg-entry/route.ts
│   ├── keys/
│   │   ├── permissions/route.ts
│   │   └── route.ts
│   ├── positions/route.ts
│   ├── realized-pnl/route.ts
│   └── spot-realized-pnl/route.ts
├── market/global/route.ts
├── v2/
│   ├── binance/
│   │   ├── depth/route.ts
│   │   ├── funding/route.ts
│   │   ├── klines/route.ts
│   │   ├── price/route.ts
│   │   └── ticker/route.ts
│   └── coingecko/global/route.ts
├── wallet/balance/route.ts
├── forgot-password/page.tsx
├── learn/                                # Eğitim içerikleri
├── pricing/page.tsx
├── reset-password/page.tsx
├── terminal/                             # Ana trading terminali
│   ├── home/
│   ├── layout.tsx
│   ├── market-intelligence/
│   ├── news/
│   ├── reporting/
│   ├── settings/api-keys/page.tsx
│   ├── staking/
│   ├── trade/
│   └── wallet/
├── verify-email/page.tsx
├── layout.tsx                            # Root layout
├── not-found.tsx
├── page.tsx                             # Landing page
├── components/                          # React bileşenleri
│   ├── auth/                            # Auth UI (LoginForm, SignupForm, UserMenu,
AuthModal)
│   ├── landing/                          # Landing page bölümleri
│   ├── providers/
│   │   └── AuthProvider.tsx
│   ├── terminal/
│   │   ├── home/
│   │   ├── layout/
│   │   ├── market-intelligence/
│   │   ├── news/
│   │   ├── personalized-dashboard/      # Modül bileşenleri
│   │   ├── reporting/
│   │   ├── staking/
│   │   ├── trade/
│   │   └── wallet/

```

```

├── ui/                                # Temel UI primitifleri
├── data/                              # Statik veri dosyaları
├── hooks/                             # Custom React hooks
├── lib/                               # Servis katmanı & yardımcılar
│   ├── api/binance/                 # Binance API istemcisi
│   ├── config/                     # API konfigürasyonu
│   ├── personalized-dashboard/      # Modül sistemi
│   ├── services/                   # Core business logic
│   ├── utils/
│   ├── auth.ts                     # NextAuth konfigürasyonu
│   ├── db.ts                       # Prisma client singleton
│   ├── email.ts                    # Resend email servisi
│   ├── encryption.ts               # AES-256-GCM şifreleme
│   └── utils.ts
├── prisma/
│   ├── migrations/
│   └── schema.prisma                # Tüm DB şeması
├── public/
├── scripts/
├── services/
│   └── WebSocketService.ts          # Multi-exchange WebSocket istemcisi
├── store/                            # Zustand store'ları
├── types/
│   └── next-auth.d.ts
├── .env
├── next.config.ts
├── package.json
├── prisma.config.ts
└── tsconfig.json

```

3. Frontend Mimarisi

3.1 Next.js App Router

Tresaurio, **Next.js App Router** kullanır. Tüm sayfalar **app/** altında tanımlanır.

Route Yapısı:

URL	Durum	Açıklama
/	Public	Landing page — oturum varsa /terminal/home 'a yönlendirir
/login, /signup	Public	Auth sayfaları
/verify-email, /forgot-password, /reset-password	Public	Auth akışı

URL	Durum	Açıklama
/terminal/home	Korunan	Trading terminali ana sayfası
/terminal/market-intelligence	Korunan	Piyasa verisi & borsa karşılaştırması
/terminal/news	Korunan	Haber akışı & duygu analizi
/terminal/settings/api-keys	Korunan	Exchange API key yönetimi
/terminal/reporting	Korunan	Analitik & raporlama (stub)
/terminal/trade, /terminal/wallet, /terminal/staking	Korunan	Gelecek özellikler (stub)
/(workspace)/personalized-dashboard	Korunan	Modüler dashboard canvas'ı
/pricing	Public	Fiyatlandırma sayfası
/learn	Public	Eğitim içerikleri
/admin/*	Şifreli	Admin paneli

Layout Stratejisi:

- **Root layout** (`app/layout.tsx`): `AuthProvider` + Geist font + global CSS
- **Terminal layout** (`app/terminal/layout.tsx`): Sidebar + header
- **Workspace layout** (`app/(workspace)/layout.tsx`): Tam ekran canvas görünümü

3.2 Server Component vs Client Component

Server Components (default, "use client" yoktur):

- `app/page.tsx` — session kontrolü + yönlendirme
- `app/terminal/home/page.tsx` — session fetch

Client Components ("use client" ile):

- Tüm interaktif bileşenler (formlar, dashboard modülleri)
- Tüm hooks (`useExchangeKeys`, `useTicker`, `useOrderBook` vb.)
- Tüm Zustand store kullanan bileşenler

3.3 Styling

Araç	Versiyon	Kullanım
Tailwind CSS	v4	Utility-first class'lar
PostCSS	v4	CSS işleme
Geist Sans	—	Ana font
Geist Mono	—	Teknik/kod metinleri

Renk Paleti:

- Arka plan: #031A1C, #0d0f14
- Primary dark: tam ekran dark theme
- Accent: borsa verilerine göre yeşil/kırmızı

3.4 UI Component Library

- **Shadcn/ui** (`components.json` konfigüre edilmiş)
- `components/ui/button.tsx` mevcut, genişletmeye hazır
- Shadcn bileşenleri ihtiyaç halinde eklenebilir

4. State Management — Zustand

Tüm store'lar `store/` dizininde, Zustand v5 kullanır.

4.1 personalizedDashboardStore.ts

En kapsamlı store. Dashboard canvas'ının tüm state'ini yönetir.

State:

```
// Canvas navigasyonu
zoom: number
panX: number
panY: number

// Modüller
modules: ModuleInstance[]
activeModuleId: string | null

// UI durumu
notesOpen: boolean
sidebarOpen: boolean
addToolOpen: boolean
templatesOpen: boolean
userMenuOpen: boolean

// Veri
notes: Note[]
alerts: AlertItem[]
favorites: string[]
templates: DashboardTemplate[]

// Kilit sistemi
lockedModules: Set<ModuleId>
```

Key Actions:

- `addModule()`, `removeModule()`, `updateModule()` — modül CRUD
- `setZoom()`, `setPan()` — canvas navigasyonu

- `lockModule()`, `unlockModule()` — hareket kilidi
- `loadFromDB()` — veritabanından yükle
- `saveToDB()` — veritabanına kaydet (3 saniye debounce)
- `saveTemplate()`, `loadTemplate()`, `deleteTemplate()` — template yönetimi

Kalıcılık:

- `localStorage` — anlık durum (hydration)
- PostgreSQL (`/api/dashboard`) — bulut yedek, 3sn debounce ile

Canvas Boyutu: 8000×4500 pixel, minimum zoom viewport'u dolduracak şekilde hesaplanır.

4.2 portfolioStore.ts

```
spotPositions: SpotPosition[]  
futuresPositions: FuturesPosition[]
```

Actions: `addPosition`, `removePosition`, `updatePosition`, `syncPrices`

Kalıcılık: `localStorage` (Zustand persist middleware)

4.3 priceStore.ts

```
prices: Record<string, number> // symbol -> fiyat  
updatedAt: number | null
```

Actions: `setPrices()`, `updatePrice()`, `resetPrices()`

4.4 orderBookStore.ts

Birden fazla modül instance'ını destekler.

```
instances: Record<string, OrderBookData>
```

Actions: `setOrderBook()`, `setSymbol()`, `getInstanceData()`

Her dashboard modülü kendi bağımsız order book state'ine sahip olabilir.

4.5 alertStore.ts

```
alerts: PriceAlert[]
```

Actions: `addAlert()`, `removeAlert()`, `checkAlerts()`
`dashboardNotificationStore` ile entegre — tetiklenince toast gösterir.

4.6 dashboardNotificationStore.ts

```
notifications: DashboardNotification[] // toast benzeri
```

Types: `"success" | "error"`

Actions: `push()`, `remove()`

4.7 exchangeFlowStore.ts

```
events: ExchangeFlowEvent[] // son 50 event
```

Actions: `addEvent()`, `clear()`

4.8 futuresPositionStore.ts

```
positions: FuturesPosition[]
```

Actions: `addPosition()`, `removePosition()`, `updatePosition()`

4.9 whaleStore.ts

```
transfers: WhaleTransfer[] // son 20 ile sınırlı
```

Actions: `addTransfer()`

5. Backend — API Routes

Tüm route'lar `app/api/` altında, Next.js Route Handler'ları ile tanımlanmış.

5.1 Standart Response Formatı

```
// Başarı
{ "success": true, "data": {...} }
```

```
// Hata
{ "success": false, "error": "Mesaj", "code": "HATA_KODU" }
```

HTTP Status Kodları:

- 200 OK
- 201 Created
- 400 Bad Request (eksik parametre, validasyon hatası)
- 401 Unauthorized (session yok, geçersiz key)
- 403 Forbidden (IP whitelist dışı)
- 409 Conflict (duplicate label)
- 501 Not Implemented (desteklenmeyen borsa)
- 500 Server Error

5.2 Auth Route'ları

Method	Endpoint	Açıklama
POST	/api/auth/signup	Kayıt — email+şifre ile kullanıcı oluşturur
POST	/api/auth/forgot-password	Şifre sıfırlama maili gönderir
POST	/api/auth/reset-password	Token ile şifre günceller
POST	/api/auth/verify-email	Email doğrular
GET/POST	/api/auth/[...nextauth]	NextAuth handler (Google OAuth + Credentials)

5.3 Exchange API Key Route'ları

Method	Endpoint	Açıklama
GET	/api/exchange/keys?exchange=binance	Kullanıcının key'lerini listeler
POST	/api/exchange/keys	Yeni key ekler (test + permission tespiti yapar)
PATCH	/api/exchange/keys	Key label'ını günceller
DELETE	/api/exchange/keys?id={id}	Key siler
POST	/api/exchange/keys/permissions	Permission'ları yeniler

5.4 Exchange Hesap Verisi (Oturum gerekli)

Method	Endpoint	Açıklama
GET	/api/exchange/account?exchange=binance	Spot bakiyeler
GET	/api/exchange/positions?exchange=binance	Futures pozisyonlar
GET	/api/exchange/avg-entry?exchange=binance	Ortalama giriş fiyatı

Method	Endpoint	Açıklama
GET	/api/exchange/realized-pnl?exchange=binance	Futures gerçekleşen P&L
GET	/api/exchange/spot-realized-pnl?exchange=binance	Spot gerçekleşen P&L

5.5 Public Piyasa Verisi (Oturum gereksiz)

Method	Endpoint	Açıklama
GET	/api/v2/binance/price?symbol=BTCUSDT	Anlık fiyat
GET	/api/v2/binance/ticker?symbols=BTC,ETH	24s ticker
GET	/api/v2/binance/depth?symbol=BTCUSDT&limit=20	Order book
GET	/api/v2/binance/klines?symbol=BTCUSDT&interval=1h	Candlestick
GET	/api/v2/binance/funding?symbols=BTCUSDT	Funding rate
GET	/api/v2/coingecko/global	Global market cap (proxy)
GET	/api/market/global	Global piyasa verisi

5.6 Dashboard Kalıcılığı

Method	Endpoint	Açıklama
GET	/api/dashboard	Kullanıcının kayıtlı layout'unu çeker
POST	/api/dashboard	Layout'u kaydeder
GET	/api/dashboard/favorites	Favori modülleri getirir
POST	/api/dashboard/favorites	Favorileri kaydeder
GET	/api/dashboard/templates	Template listesi
POST	/api/dashboard/templates	Yeni template kaydeder
GET	/api/dashboard/templates/{id}	Belirli template'i yükler
DELETE	/api/dashboard/templates/{id}	Template siler

5.7 Diğer Route'lar

Method	Endpoint	Açıklama
GET	/api/wallet/balance? address=0x...&chain=ethereum	Multi-chain cüzdan bakiyesi
POST	/api/analytics/pnl	P&L analitik

Method	Endpoint	Açıklama
GET	/api/admin/users	Kullanıcı listesi (x-admin-password header gerekli)

6. Veritabanı — Prisma & PostgreSQL

ORM: Prisma v7.3.0

Adapter: @prisma/adapter-pg (connection pooling)

DB: PostgreSQL

6.1 User & Auth Modelleri

```

model User {
  id          String    @id @default(cuid())
  name        String?
  email       String    @unique
  emailVerified DateTime?
  password    String?   // OAuth kullanıcıları için null
  image       String?
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt

  accounts    Account[]
  sessions    Session[]
  apiKeys     ApiKey[]
  dashboard   UserDashboard?
  templates   DashboardTemplate[]
  favorites   UserFavorite[]
  activityLogs UserActivityLog[]
}

model Account {
  id          String    @id @default(cuid())
  userId      String
  type        String
  provider     String    // "google"
  providerAccountId String
  refresh_token String? @db.Text
  access_token String? @db.Text
  expires_at   Int?
  token_type   String?
  scope        String?
  id_token     String? @db.Text
  session_state String?

  user User @relation(fields: [userId], references: [id], onDelete: Cascade)
  @@unique([provider, providerAccountId])
}

model Session {

```

```

    id          String    @id @default(cuid())
    sessionToken String    @unique
    userId       String
    expires      DateTime

    user User @relation(fields: [userId], references: [id], onDelete: Cascade)
}

model VerificationToken {
    identifier String
    token       String    @unique
    expires     DateTime

    @@unique([identifier, token])
}

model UserActivityLog {
    id          String    @id @default(cuid())
    userId      String
    action      String      // "login", "logout", vb.
    ipAddress   String?
    userAgent   String?
    createdAt   DateTime @default(now())

    user User @relation(fields: [userId], references: [id], onDelete: Cascade)
    @@index([userId])
    @@index([createdAt])
}

```

6.2 API Key & Trading Modelleri

```

model ApiKey {
    id          String    @id @default(cuid())
    userId      String
    exchange    String      // binance | okx | bybit | coinbase | hyperliquid
    label       String?
    apiKey      String      // AES-256-GCM şifreli
    apiSecret   String      // AES-256-GCM şifreli
    passphrase  String?     // Şifreli (OKX, Coinbase için)
    permissions String[]    // ["spot", "futures", "withdraw"]
    isTestnet   Boolean    @default(false)
    isActive    Boolean    @default(true)
    lastUsedAt  DateTime?
    createdAt   DateTime @default(now())
    updatedAt   DateTime @updatedAt

    user User @relation(fields: [userId], references: [id], onDelete: Cascade)
    @@unique([userId, exchange, label])
    @@index([userId])
    @@index([exchange])
    @@index([isActive])
}

```

```
}

model Trade {
  id          String    @id @default(cuid())
  exchange    String
  symbol      String
  side        String     // buy | sell
  type        String     // market | limit
  quantity    Float
  price       Float
  quoteQty    Float
  commission  Float?
  commissionAsset String?
  orderId     String
  tradeId     String?
  isMaker     Boolean    @default(false)
  executedAt  DateTime
  createdAt   DateTime @default(now())

  @@unique([exchange, orderId])
  @@index([exchange, symbol])
  @@index([executedAt])
}

model Order {
  id          String    @id @default(cuid())
  exchange    String
  symbol      String
  orderId     String
  clientOrderId String?
  side        String     // buy | sell
  type        String     // market | limit | stop_limit
  status      String     // open | filled | cancelled | expired
  quantity    Float
  price       Float?
  stopPrice   Float?
  filledQty   Float      @default(0)
  avgPrice    Float?
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  @@unique([exchange, orderId])
  @@index([exchange, symbol])
  @@index([status])
}

model PortfolioSnapshot {
  id          String    @id @default(cuid())
  totalValueUSD Float
  spotValueUSD Float
  futuresValueUSD Float
  unrealizedPnL Float
  realizedPnL  Float
  snapshotAt   DateTime @default(now())
}
```



```

    @@index([snapshotAt])
  }

```

6.3 Dashboard Modelleri

```

model UserDashboard {
  id          String   @id @default(cuid())
  userId      String   @unique
  layout      Json      // modules, zoom, panX, panY, notes, alerts,
lockedModules
  updatedAt   DateTime @updatedAt

  user User @relation(fields: [userId], references: [id], onDelete: Cascade)
  @@index([userId])
}

model DashboardTemplate {
  id          String   @id @default(cuid())
  userId      String
  name        String
  layout      Json      // { modules, zoom, panX, panY, lockedModules,
canvasWidth, canvasHeight }
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  user User @relation(fields: [userId], references: [id], onDelete: Cascade)
  @@unique([userId, name])
  @@index([userId])
}

model UserFavorite {
  id          String   @id @default(cuid())
  userId      String
  type        String   // modül tipi adı
  favoritedAt DateTime @default(now())

  user User @relation(fields: [userId], references: [id], onDelete: Cascade)
  @@unique([userId, type])
  @@index([userId])
}

```

6.4 DB Notları

- Tüm user ilişkilerinde **onDelete: Cascade** — kullanıcı silinince tüm verileri gider
- Dashboard layout'u **Json** tipinde — esnek şema
- Şifreli API key'ler veritabanında AES-256-GCM formatında saklanır
- **UserActivityLog** güvenlik audit trail için

7. Servisler & Kütüphaneler

7.1 lib/services/apiKeyService.ts

Exchange API key'lerinin veritabanına şifreli kaydedilmesini, çekilmesini ve yönetilmesini sağlar.

Fonksiyonlar:

Fonksiyon	Açıklama
<code>saveApiKey()</code>	Şifreler ve kaydeder
<code>getApiKeys()</code>	Listeler (secret dönmez)
<code>getDecryptedApiKey()</code>	API çağrısı için şifre çözer (sadece sunucu tarafı)
<code>updateApiKeyLabel()</code>	Label günceller
<code>updateApiKeyPermissions()</code>	Permission listesi günceller
<code>deleteApiKey()</code>	Hard delete
<code>deactivateApiKey()</code>	Soft delete (isActive: false)
<code>testApiKey()</code>	Borsa ile key'i doğrular

7.2 lib/services/exchangeService.ts

Birden fazla borsayı tek arayüzle soyutlar.

Desteklenen Borsalar: Binance, OKX, Bybit, Coinbase, Hyperliquid

Fonksiyonlar:

Fonksiyon	Açıklama
<code>getSpotBalances()</code>	Spot cüzdan bakiyesi
<code>getFuturesPositions()</code>	Açık futures pozisyonları
<code>getFuturesAvailableBalance()</code>	Margin/teminat bakiyesi
<code>getFuturesRealizedPnl()</code>	Gerçekleşen futures karı
<code>getSpotRealizedPnl()</code>	Spot işlem geçmişi
<code>getAverageEntryPrice()</code>	Ortalama giriş analizi
<code>detectApiKeyPermissions()</code>	Key yeteneklerini test eder
<code>hasApiKey()</code>	Kullanıcının o borsa için key'i var mı?

Implementation Detayları:

- Binance: HMAC-SHA256 imza
- Sunucu saat senkronizasyonu (imza doğrulaması için)

- Hata kodları: `INVALID_API_KEY`, `IP_NOT_WHITELISTED`, `INSUFFICIENT_PERMISSIONS`, vb.

7.3 lib/services/exchangeFetch.ts

Tüm borsa API isteklerini saran HTTP proxy katmanı.

İki Mod:

1. Direct (Yerel geliştirme):

```
Next.js → fetch() → Exchange API
```

2. Proxy (Production/Vercel):

```
Next.js → HTTP CONNECT tunnel → VPS → Exchange API
```

Vercel'de statik IP zorunluluğu için (borsa IP whitelist'i desteklemek amacıyla).

Environment Variables:

- `EXCHANGE_EGRESS_MODE` — "proxy" veya boş
- `EXCHANGE_EGRESS_URL` — `http://vps-ip:3128`
- `EXCHANGE_EGRESS_TOKEN` — Bearer token

Özellikler:

- 15 saniye timeout
- Proxy hatalarında otomatik retry
- API key'ler proxy'e iletilmez (TLS tüneli içinde)
- `undici` kütüphanesi ile CONNECT proxy desteği

7.4 lib/api/binance/BinanceService.ts

Merkezi Binance istemcisi.

Özellikler:

- In-memory LRU cache (TTL destekli)
- Otomatik rate limiting
- Timeout yönetimi (yapılandırılabilir)
- Exponential backoff ile retry

Metodlar:

Metod	Açıklama
<code>getPrice()</code>	Tek sembol fiyatı

Metod	Açıklama
<code>get24hrTickers()</code>	Çoklu sembol ticker
<code>getDepth()</code>	Order book
<code>getKlines()</code>	Candlestick verisi
<code>getPremiumIndex()</code>	Futures premium
<code>getFundingRate()</code>	Futures funding

7.5 lib/db.ts — Prisma Singleton

```
// Connection pooling ile Prisma client
import { PrismaClient } from "@prisma/client";
import { PrismaPg } from "@prisma/adapter-pg";

const pool = new Pool({ connectionString: DATABASE_URL });
const adapter = new PrismaPg(pool);
const prisma = new PrismaClient({ adapter });

// Node.js hot reload'da connection leak'i önler
if (process.env.NODE_ENV !== "production") globalThis.prisma = prisma;
```

Özellikler:

- PostgreSQL adapter
- Connection pooling (pg)
- Geliştirmede query logging
- Singleton pattern

7.6 lib/encryption.ts

Algoritma: AES-256-GCM

Şifreli format: iv:authTag:encryptedData (base64)

Fonksiyon	Açıklama
<code>encrypt(text)</code>	Şifreler, random IV üretir
<code>decrypt(encryptedText)</code>	Çözer, auth tag ile bütünlük doğrular
<code>generateEncryptionKey()</code>	32-byte hex key üretir

Key, environment variable'dan SHA256 hash ile türetilir.

7.7 lib/email.ts

Servis: Resend

Fonksiyon	Açıklama
<code>sendVerificationEmail()</code>	HTML doğrulama maili
<code>sendPasswordResetEmail()</code>	Şifre sıfırlama maili
<code>generateVerificationToken()</code>	32-byte crypto-random token

Email şablonları dark theme, Tresaurio markası.

7.8 services/WebSocketService.ts

Multi-exchange WebSocket istemcisi.

Desteklenen:

- Binance Spot: `wss://stream.binance.com:9443/ws`
- Binance Futures: `wss://fstream.binance.com/ws`
- OKX: `wss://ws.okx.com:8443/ws/v5/public`
- Bybit: `wss://stream.bybit.com/v5/public/spot|linear`
- Coinbase: `wss://ws-feed.exchange.coinbase.com`

Özellikler:

- Otomatik yeniden bağlanma (exponential backoff, max 5 deneme)
- Order book throttle (2 update/sn)
- REST API fallback (WebSocket başarısızsa)
- Çoklu eş zamanlı bağlantı

8. Authentication Sistemi

Kütüphane: NextAuth.js v5

Session Stratejisi: JWT

Adapter: Prisma

8.1 Provider'lar

1. Google OAuth:

Google → code → token exchange → kullanıcı bul veya oluştur → session

2. Credentials (email + şifre):

email+şifre → bcrypt.compare (12 round) → user objesi → JWT session

8.2 Kayıt Akışı

1. Kullanıcı email + şifre gönderir
2. Email unique mi kontrol et
3. Şifre ≥ 8 karakter validasyonu
4. bcrypt ile hash (12 round)
5. `prisma.user.create()`
6. `emailVerified` anında set edilir (ayrı doğrulama adımı yok)

8.3 Şifre Sıfırlama Akışı

1. `/forgot-password` → email gir
2. `generateVerificationToken()` → `VerificationToken` tablosuna kaydet
3. Resend ile link gönder
4. `/reset-password?token=...` → bcrypt ile yeni şifre hash'le
5. `user.password` güncelle, token'ı sil

8.4 Session Erişimi

Server tarafında (API routes):

```
const session = await auth();
if (!session?.user?.id) {
  return NextResponse.json({ error: "Unauthorized" }, { status: 401 });
}
```

Client tarafında:

```
const { data: session } = useSession();
```

8.5 JWT Callback

```
// user.id token'a eklenir
jwt: async ({ token, user }) => {
  if (user) token.id = user.id;
  return token;
}

// session objesine id eklenir
session: async ({ session, token }) => {
  session.user.id = token.id as string;
  return session;
}
```

8.6 Activity Logging

Her login ve logout olayı `UserActivityLog` tablosuna kaydedilir:

- `action, ipAddress, userAgent, createdAt`

9. Dış Entegrasyonlar

9.1 Exchange REST API'leri

Borsa	Base URL	Auth Tipi
Binance	https://api.binance.com	HMAC-SHA256 imza
OKX	https://www.okx.com/api	API key + imza
Bybit	https://api.bybit.com	API key + imza
Coinbase	https://api.coinbase.com	API key + imza
Hyperliquid	Decentralized	Sadece API key

9.2 Exchange WebSocket'leri

Borsa	Spot URL	Futures URL
Binance	wss://stream.binance.com:9443/ws	wss://fstream.binance.com/ws
OKX	wss://ws.okx.com:8443/ws/v5/public	Aynı
Bybit	wss://stream.bybit.com/v5/public/spot	.../linear
Coinbase	wss://ws-feed.exchange.coinbase.com	—

9.3 Piyasa Verisi API'leri

CoinGecko (proxy üzerinden):

- <https://api.coingecko.com/api/v3/global>
- Global market cap, BTC/ETH dominance, 24s hacim

CryptoCompare (direkt `useNews` hook'undan):

- <https://min-api.cryptocompare.com/data/v2/news>
- Haber akışı, kategori filtresi

9.4 Blockchain RPC Endpoint'leri

Ethereum (fallback zinciri):

1. <https://rpc.ankr.com/eth>
2. <https://eth.llamarpc.com>
3. <https://cloudflare-eth.com>

BSC:

1. <https://rpc.ankr.com/bsc>
2. <https://bsc-dataseed1.binance.org>
3. <https://bsc-dataseed2.binance.org>

Tron:

- <https://apilist.tronscan.org/api/account>

Solana:

1. <https://rpc.ankr.com/solana>
2. <https://solana-mainnet.rpc.extrnode.com>
3. <https://api.mainnet-beta.solana.com>

9.5 Email Servisi — Resend

- `sendVerificationEmail()` — doğrulama linki
- `sendPasswordResetEmail()` — şifre sıfırlama
- From: onboarding@resend.dev (sandbox)

10. Custom Hooks

Tüm hook'lar `hooks/` dizininde, "`use client`" direktifi ile.

10.1 useExchangeKeys.ts

Exchange API key yönetimi.

```
const {
  keys, loading, error, refetch,
  deleteKey, updateLabel, refreshPermissions,
  hasKeyForExchange, getKeysForExchange
} = useExchangeKeys();
```

Fetch Çağrıları:

- `GET /api/exchange/keys` — listele
- `DELETE /api/exchange/keys?id={id}` — sil
- `PATCH /api/exchange/keys` — label güncelle
- `POST /api/exchange/keys/permissions` — permission yenile

10.2 useTicker.ts

WebSocket tabanlı 24s ticker. WebSocket başarısızsa REST'e düşer.


```
const { data, loading, error, status, retry } = useTicker({
  symbol: "BTCUSDT",
  exchange: "binance",
  marketType: "spot"
});
```

Status: `connecting` | `connected` | `error` | `fallback`

10.3 useOrderBook.ts

WebSocket tabanlı order book. Spread, mid price hesaplar.

```
const {
  bids, asks, spread, spreadPercent, midPrice,
  loading, error, status, retry, lastUpdate
} = useOrderBook({ symbol, exchange, limit: 20 });
```

- Bybit/Coinbase için incremental update desteği
 - Throttle: 2 update/sn
-

10.4 useKlines.ts

Candlestick verisi. Doğrudan borsa API'lerine fetch atar (proxy değil).

Desteklenen Borsalar:

- <https://api.binance.com/api/v3/klines>
 - <https://www.okx.com/api/v5/market/candles>
 - <https://api.bybit.com/v5/market/kline>
 - <https://api.exchange.coinbase.com/products/.../candles>
-

10.5 usePnL.ts

Portföy P&L hesaplama.

Döndürür:

- Spot: unrealized, investment, currentValue
 - Futures: unrealized, margin, value
 - Total: PnL %, performans metrikleri
-

10.6 useNews.ts

CryptoCompare haber akışı.

```
const { news, loading, error, settings, updateSettings, refresh } = useNews();
```

Özellikler:

- Kategori filtresi: Bitcoin, Ethereum, DeFi, NFT, Regulation
- Ayarlar localStorage'da saklanır

10.7 useLastOrders.ts

Bağlı borsadan işlem geçmişi. Sayfalama desteği.

11. Personalized Dashboard Modül Sistemi

11.1 Canvas Mimarisi

Özellik	Değer
Canvas boyutu	8000 × 4500 px
Max zoom	2x
Min zoom	Viewport'u dolduracak şekilde hesaplanır
Pan	Serbest kaydırma
Kayıt	localStorage + PostgreSQL (3sn debounce)

11.2 Modül Yapısı

```
interface ModuleInstance {
  id: string;           // UUID
  type: string;         // Modül tipi
  title: string;        // Görünen isim
  category: ModuleCategory;
  x: number;            // Canvas X konumu (px)
  y: number;            // Canvas Y konumu (px)
  width: number;        // Genişlik (px)
  height: number;       // Yükseklik (px)
  minimized?: boolean;
  contentZoom?: number;
}
```

11.3 Modül Kategorileri ve Tipleri

Piyasa Verisi:

- **live-prices** — Canlı fiyat ticker'ı
- **order-book** — Gerçek zamanlı derinlik

- [funding-rate](#) — Futures funding görselleştirme
- [exchange-comparison](#) — Borsa karşılaştırması

Portföy:

- [spot-positions](#) — Spot varlıklar
- [futures-positions](#) — Açık pozisyonlar
- [pnl-overview](#) — P&L özeti
- [pnl-analysis](#) — Detaylı breakdown

Flow & Intelligence:

- [exchange-flow](#) — Yatırım/çekim aktivitesi
- [whale-alerts](#) — Büyük transfer tespiti
- [token-flow](#) — Token hareketi analizi
- [exchange-netflow](#) — BTC/ETH giriş/çıkış

Analitik:

- [liquidity-analysis](#) — Order book kalitesi
- [slippage-monitor](#) — İşlem kayması
- [spread-monitor](#) — Spread analizi
- [rsi-heatmap](#) — Teknik göstergeler
- [market-efficiency](#) — Piyasa kalite skoru

Haber & Veri:

- [news-feed](#) — CryptoCompare haber akışı

Uyarılar:

- [active-alerts](#) — Aktif fiyat uyarıları
- [create-alert](#) — Yeni uyarı oluşturma

Trading:

- [spot-actions](#) — Al/sat arayüzü
- [futures-actions](#) — Kaldıraçlı işlem
- [last-orders](#) — Son emirler
- [risk-calculator](#) — Pozisyon boyutlandırma

Etkinlikler:

- [token-unlock](#) — Vesting takibi
- [ico-calendar](#) — Yaklaşan ICO'lar
- [reward-calendar](#) — Staking ödülleri

Araçlar:

- [chart](#) — Grafikler
- [wallet-inspector](#) — Adres bakiye sorgulama
- [dca-calculator](#) — Dollar-cost averaging

- **fee-structure** — Borsa ücret karşılaştırması

11.4 Bir Modül Nasıl Çalışır? (Yaşam Döngüsü)

Bir modülün ekranda görünmesinden veri çekip göstermesine kadar olan akış:

1. KAYIT (lib/personalized-dashboard/moduleRegistry.ts)

Her modül tipi registry'de tanımlıdır:

```
{
  type: "futures-positions",
  title: "Futures Pozisyonlar",
  category: "portfolio",
  defaultWidth: 600,
  defaultHeight: 400,
  component: FuturesPositionsModule ← React bileşeni
}
```

2. CANVAS'A EKLEME

Kullanıcı sidebar'dan modül seçer

- personalizedDashboardStore.addModule(type)
- modules[] dizisine ModuleInstance eklenir
- { id, type, x, y, width, height }
- Canvas bu listeyi map'ler, her ModuleInstance için registry'deki component'i render eder

3. RENDER (components/terminal/personalized-dashboard/{Name}.tsx)

Modül bileşeni mount olur ("use client")

Her modül kendi veri stratejisini seçer:

A) WebSocket tabanlı (gerçek zamanlı):

- useOrderBook() veya useTicker() hook'u çağrılır
- WebSocket açılır (borsaya direkt, tarayıcıdan)
- Veri gelince priceStore / orderBookStore güncellenir
- Bileşen store'u dinler, her güncellemede re-render

B) REST API tabanlı (periyodik fetch):

- useEffect içinde fetch("/api/exchange/positions")
- Next.js API route → exchangeService → borsa API → JSON
- useState ile component state'e set edilir
- Interval ile tekrar çekilebilir

C) Zustand store tabanlı (paylaşımlı state):

- portfolioStore, alertStore vb. doğrudan okunur
- Başka bir bileşen store'u güncellediğinde otomatik re-render
- Fetch yok, store zaten doluysa anında görünür

D) Statik / mock veri:

- Bazı modüller henüz gerçek veri bağlamadı
- Sabit data ile render olur (market-efficiency, vb.)

4. KONUM & BOYUT YÖNETİMİ

Kullanıcı modülü taşır veya yeniden boyutlandırır

- `personalizedDashboardStore.updateModule(id, { x, y, width, height })`
- Canvas re-render (sadece o modül)
- `localStorage`'a anında yazılır
- 3 saniye debounce → `POST /api/dashboard` → PostgreSQL

5. KAYIT & YÜKLEME

Sayfa ilk yüklendiğinde:

- `personalizedDashboardStore.loadFromDB()`
- `GET /api/dashboard` → Prisma → `userDashboard.layout` (JSON)
- `modules[], zoom, panX, panY, lockedModules` restore edilir
- `localStorage` yoksa DB'den, DB yoksa `defaultModules`'dan başlar

6. TEMPLATE SİSTEMİ

Kullanıcı "template kaydet" der

- Mevcut layout (`modules, zoom, pan`) snapshot'lanır
- `POST /api/dashboard/templates` → Prisma → `DashboardTemplate`
- İsim ile kaydedilir, sonraki oturumda yüklenebilir

11.5 Modül Veri Kaynağı Referansı

Her modülün veriyi nereden aldığı:

Modül	Veri Kaynağı	Protokol
<code>live-prices</code>	<code>useTicker</code> hook	WebSocket → borsa
<code>order-book</code>	<code>useOrderBook</code> hook	WebSocket → borsa
<code>funding-rate</code>	<code>/api/v2/binance/funding</code>	REST (fetch)
<code>futures-positions</code>	<code>/api/exchange/positions</code>	REST (fetch)
<code>spot-positions</code>	<code>portfolioStore</code>	Zustand store
<code>pnl-overview</code>	<code>usePnL</code> hook + <code>portfolioStore</code>	Store hesaplama
<code>exchange-flow</code>	<code>exchangeFlowStore</code>	Zustand store
<code>whale-alerts</code>	<code>whaleStore</code>	Zustand store
<code>news-feed</code>	<code>useNews</code> hook	REST → CryptoCompare (direkt)
<code>active-alerts</code>	<code>alertStore</code>	Zustand store
<code>wallet-inspector</code>	<code>/api/wallet/balance</code>	REST (fetch)
<code>chart</code>	<code>useKlines</code> hook	REST → borsa (direkt)

11.6 Modül Özellikleri

Özellik	Açıklama
Sürükle	Canvas'ta serbestçe taşı
Yeniden boyutlandır	Kenarları sürükle
Kilitle	Yanlışlıkla hareketi engelle
Swap	Bir modülü diğerinin üstüne sürükle → yer değiştir
Boyut kopyala	Bir modülün boyutunu diğerine uygula
Favoriler	Sık kullanılan modüllere hızlı erişim
Template	Dashboard layout'unu kaydet ve yükle
Minimize	Modülü küçült

12. TypeScript Tipleri

12.1 Auth Tipi Genişletmesi

```
// types/next-auth.d.ts
declare module "next-auth" {
  interface Session {
    user: {
      id: string;
    } & DefaultSession["user"];
  }
}
```

12.2 Ana Tipler ([lib/personalized-dashboard/types.ts](#))

Kategori	Tipler
Modül Sistemi	ModuleId , ModuleCategory , ModuleType , ModuleInstance
Pozisyonlar	SpotPosition , FuturesPosition , FuturesSide
Portföy	PnLBreakdown , PnLTimeframe
Flow	WhaleTransfer , ExchangeFlowEvent , TokenFlowEvent
Uyanlar	AlertCondition , AlertItem , PriceAlert
Haber	NewsItem , NewsSentiment
Emirler	OrderSide , OrderType , SpotOrder , FuturesOrder , OrderHistory
Analitik	LiquidityMetrics , SlippageData , MarketEfficiencyMetrics
Cüzdan	WalletData , TokenBalance , WalletTransaction

Kategori	Tipler
Staking	<code>StakePosition</code> , <code>RewardEvent</code>
Diğer	<code>ICOEvent</code> , <code>TokenUnlock</code> , <code>VestingSchedule</code> , <code>ETFFlow</code> , <code>RSIData</code>

12.3 Hook Tipleri

```
// useOrderBook
interface OrderBookLevel { price: number; quantity: number; }
interface OrderBookData { bids: OrderBookLevel[]; asks: OrderBookLevel[]; spread:
number; spreadPercent: number; midPrice: number; }
interface UseOrderBookOptions { symbol: string; marketType: "spot" | "futures";
exchange: string; limit?: number; enabled?: boolean; timeoutMs?: number; }

// useTicker
interface TickerData { symbol: string; lastPrice: number; priceChange: number;
priceChangePercent: number; volume: number; /* ... */ }
interface UseTickerOptions { symbol: string; marketType: "spot" | "futures";
exchange: string; enabled?: boolean; timeoutMs?: number; }
```

13. Environment Variables

```
# -----
# Veritabanı
# -----
DATABASE_URL="postgresql://user:pass@host/db?sslmode=require"

# -----
# NextAuth.js
# -----
AUTH_SECRET="super-gizli-32-karakter-key"
AUTH_URL="http://localhost:3000"
AUTH_TRUST_HOST=true

# -----
# Google OAuth
# -----
GOOGLE_CLIENT_ID="..."
GOOGLE_CLIENT_SECRET="..."

# -----
# Email (Resend)
# -----
RESEND_API_KEY="re_..."

# -----
# Şifreleme (API key'ler için)
# -----
```

```
ENCRYPTION_KEY="64-karakter-hex-key"    # 32 byte

# -----
# Exchange Proxy (Sadece production/Vercel)
# -----
EXCHANGE_EGRESS_MODE="proxy"             # "proxy" veya boş
EXCHANGE_EGRESS_URL="http://vps:3128"
EXCHANGE_EGRESS_TOKEN="64-karakter-hex"

# -----
# Admin Paneli
# -----
ADMIN_PASSWORD="..."                   # /api/admin/users için
```

Zorunlu:

- DATABASE_URL
- AUTH_SECRET
- GOOGLE_CLIENT_ID + GOOGLE_CLIENT_SECRET
- RESEND_API_KEY
- ENCRYPTION_KEY

Opsiyonel (Sadece production):

- EXCHANGE_EGRESS_* — Vercel'de statik IP için
- ADMIN_PASSWORD — Admin endpoint koruması

14. Deployment & Altyapı

14.1 Build Süreci

```
# Geliştirme
npm run dev          # next dev

# Production build
npm run build        # prisma generate && next build

# Production start
npm start            # next start
```

14.2 Prisma Workflow

```
# Şema değişikliği sonrası
npx prisma migrate dev --name "migration_name"

# Production deployment
npx prisma migrate deploy
```



```
# Prisma client yenile
npx prisma generate

# DB'yi görüntüle
npx prisma studio
```

14.3 Hedef Platformlar

Vercel (Birincil):

- Next.js native host
- Otomatik deploy (git push)
- `EXCHANGE_EGRESS_*` env variables → statik IP için VPS proxy
- Serverless function limitleri (15sn timeout)

Self-hosted Node.js:

- `npm run build && npm start`
- Reverse proxy (nginx/caddy)
- PM2 process manager

Docker:

- Dockerfile mevcut (proje root'unda)
- `docker-compose` ile çalıştırılabilir
- Port: 3000

14.4 Veritabanı Deployment

Uyumlu servisler:

- Neon (serverless PostgreSQL)
- Supabase
- AWS RDS / Aurora
- PlanetScale (uyumluluk modunda)
- Self-hosted PostgreSQL

Bağlantı havuzu: pg Pool, 20 bağlantı, 30-60sn keep-alive

14.5 Middleware

`middleware.ts` — Route koruması ve session kontrolü:

- Korunan route'lara giriş → session yoksa `/login`'e yönlendir
- Oturum açık + landing page → `/terminal/home`'a yönlendir

15. Güvenlik

15.1 Authentication

- **Session:** NextAuth JWT, secure HttpOnly cookie
- **Şifre:** bcryptjs, 12 salt round
- **OAuth:** Google OAuth, `allowDangerousEmailAccountLinking: true`
- **Activity Log:** Her login/logout DB'ye kaydedilir

15.2 API Key Güvenliği

- API key'ler veritabanında AES-256-GCM şifreli saklanır
- Şifresi çözülmüş key client'a hiçbir zaman dönmez
- Exchange'e istek sadece sunucu tarafından yapılır
- Şifre çözme işlemi sadece imzalı isteğin yapılacağı anda

15.3 Request Güvenliği

- Tüm korunan route'larda `auth()` session kontrolü
- Input validasyonu tüm route handler'larda
- Admin endpoint: `x-admin-password` header token
- Exchange proxy: Bearer token ile doğrulama

15.4 Production

- `.env` dosyaları git'e commit'lenmez
- HTTPS zorunlu
- PostgreSQL SSL (`?sslmode=require`)
- Sensitive veriler loglanmaz

16. Performans Optimizasyonları

16.1 Frontend

Optimizasyon	Uygulama
Code splitting	Next.js otomatik route splitting
Font optimizasyonu	<code>next/font</code>
Image optimizasyonu	<code>next/image</code>
State caching	Zustand localStorage persist
Debounce	Dashboard kayıt: 3sn debounce

16.2 Backend

Optimizasyon	Uygulama
DB index'leri	<code>userId</code> , <code>exchange</code> , <code>status</code> , <code>createdAt</code>
Connection pooling	pg Pool (20 bağlantı)
In-memory cache	BinanceService LRU cache + TTL

Optimizasyon	Uygulama
Rate limiting	Binance rate limiter (<code>checkRateLimit</code>)
Timeout	15sn default, yapılandırılabilir

16.3 WebSocket

Optimizasyon	Uygulama
Order book throttle	2 update/sn
Reconnect	Exponential backoff, max 5 deneme
Fallback	WebSocket başarısızsa REST API
Çoklu bağlantı	Eş zamanlı multi-exchange WebSocket

17. Geliştirici Rehberi

17.1 Yerel Geliştirme Kurulumu

```
# Repo'yu klonla
git clone https://github.com/.../tesaurio.git
cd tesaurio

# Bağımlılıkları yükle
npm install

# Environment variables
cp .env.example .env.local
# .env.local dosyasını düzenle

# Veritabanını başlat
npx prisma migrate dev

# Geliştirme sunucusunu başlat
npm run dev
# http://localhost:3000
```

17.2 Yeni Modül Ekleme

- `components/terminal/personalized-dashboard/{ModuleName}.tsx` oluştur
- `lib/personalized-dashboard/types.ts` — `ModuleType` union'a ekle
- `lib/personalized-dashboard/moduleRegistry.ts` — register et
- `lib/personalized-dashboard/defaultModules.ts` — varsayılan olarak ekle (opsiyonel)

17.3 Yeni API Route Ekleme

- `app/api/{feature}/route.ts` oluştur

2. `GET`, `POST`, `DELETE`, `PATCH` fonksiyonlarını export et
3. Gerekiyorsa `const session = await auth()` ile koruma ekle
4. Standart response formatını kullan

17.4 Yeni Hook Ekleme

1. `hooks/{hookName}.ts` oluştur
2. `"use client"` direktifi ekle
3. Doğru tiplerle export et
4. `hooks/index.ts`'e ekle

17.5 Kod Standartları

Kural	Değer
Dil	TypeScript (strict mode)
Linting	ESLint 9
Styling	Tailwind CSS utility class'ları
Component tipi	Functional, interaktiflere "use client"
State	Karmaşık → Zustand, lokal → useState
API	REST + JSON response
Error handling	try-catch + açıklayıcı mesajlar

18. Microservice Migration Haritası

Bu bölüm, mevcut Next.js monolitinin 3 microservice'e ayrılma sürecini belgeler.
Yeni frontend bu microservice'lerle konuşacak — mevcut `/api/...` route'larına değil.

18.1 Mevcut vs Yeni Mimari

Mevcut (Monolith):

Frontend → `/api/...` (Next.js route handler) → Prisma → PostgreSQL
→ Borsa API'leri

Yeni (Microservices):

Frontend → api-gateway → market-data (public piyasa verisi)
→ exchange-service (kullanıcıya özel borsa verisi)
→ [diğer servisler]
→ PostgreSQL (api-gateway veya servisler üzerinden)

18.2 Mevcut API Route → Yeni Microservice Haritası

market-data servisi

Kimlik doğrulama gerektirmeyen, herkese açık piyasa verisi.

Mevcut Route	Yeni Servis	Açıklama
GET /api/v2/binance/ticker	market-data	24s ticker
GET /api/v2/binance/price	market-data	Anlık fiyat
GET /api/v2/binance/depth	market-data	Order book
GET /api/v2/binance/klines	market-data	Candlestick
GET /api/v2/binance/funding	market-data	Funding rate
GET /api/v2/coingecko/global	market-data	Global market cap
GET /api/market/global	market-data	Global piyasa

Not: `useKlines` ve `useNews` hook'ları şu an borsalara/CryptoCompare'e doğrudan fetch atıyor (Next.js'i atlatarak). Yeni frontend bu hook'ları `market-data` üzerinden yönlendirmeli mi yoksa direkt devam mı etmeli — karar verilmesi gerekiyor.

exchange-service servisi

Kullanıcının kendi borsa hesabına ait veriler. Kimlik doğrulama zorunlu.

Mevcut Route	Yeni Servis	Açıklama
GET /api/exchange/account	exchange-service	Spot bakiyeler
GET /api/exchange/positions	exchange-service	Futures pozisyonlar
GET /api/exchange/realized-pnl	exchange-service	Futures P&L
GET /api/exchange/spot-realized-pnl	exchange-service	Spot P&L
GET /api/exchange/avg-entry	exchange-service	Ortalama giriş
GET /api/exchange/keys	exchange-service	API key listesi
POST /api/exchange/keys	exchange-service	API key ekle
PATCH /api/exchange/keys	exchange-service	Label güncelle
DELETE /api/exchange/keys	exchange-service	Key sil
POST /api/exchange/keys/permissions	exchange-service	Permission yenile

Not: API key'ler mevcut sistemde AES-256-GCM şifreli PostgreSQL'de saklanıyor (`ApiKey` tablosu). `exchange-service` bu tabloyu doğrudan yönetmeli.

api-gateway servisi

Frontend'in tek muhatap olduđu giriş noktası. Auth, routing ve orchestration.

Mevcut Route	Yeni Servis	Açıklama
POST /api/auth/signup	api-gateway	Kayıt
POST /api/auth/forgot-password	api-gateway	Şifre sıfırlama
POST /api/auth/reset-password	api-gateway	Şifre güncelle
GET/POST /api/auth/verify-email	api-gateway	Email doğrulama
GET /api/dashboard	api-gateway	Dashboard layout çek
POST /api/dashboard	api-gateway	Dashboard layout kaydet
GET/POST /api/dashboard/templates	api-gateway	Template yönetimi
GET/DELETE /api/dashboard/templates/{id}	api-gateway	Tekil template
GET/POST /api/dashboard/favorites	api-gateway	Favori modüller
GET /api/wallet/balance	api-gateway	Multi-chain bakiye
POST /api/analytics/pnl	api-gateway	P&L analitik
GET /api/admin/users	api-gateway	Admin panel

18.3 Yeni Frontend İçin Kritik Notlar

Prisma tamamen kalkıyor

Mevcut Next.js frontend'i /api/... route'ları üzerinden Prisma'ya yazıyordu. Yeni frontend bunu yapmaz — her şey microservice HTTP isteğiyle yapılır.

Silinecek pattern:

```
// ESKİ: Next.js API route → Prisma
const session = await auth();
const dashboard = await prisma.userDashboard.findUnique({ where: { userId:
session.user.id } });
```

Yeni pattern:

```
// YENİ: fetch → api-gateway → servis → DB
const res = await fetch(`${API_GATEWAY_URL}/dashboard`, {
  headers: { Authorization: `Bearer ${token}` }
});
```

Auth token yönetimi değişiyor

Mevcut sistemde NextAuth JWT session cookie ile çalışıyor. Yeni frontend'in **api-gateway**'e nasıl token göndereceği netleştirilmeli:

- Bearer token (JWT)
- Cookie-based session
- API-gateway kendi session'ını mı yönetiyor?

WebSocket bağlantıları değişmiyor (muhtemelen)

useTicker, **useOrderBook** hook'ları borsalara **doğrudan** WebSocket açıyor (Binance, OKX, Bybit, Coinbase). Bu Next.js'e bağımlı değil, tarayıcıdan direkt gidiyor. Yeni frontend bu hook'ları olduğu gibi taşıyabilir.

Zustand store'ları taşınabilir

store/ dizinindeki tüm Zustand store'ları pure TypeScript — Next.js'e bağımlılığı yok. Yeni frontend reposu oluşturulduğunda olduğu gibi kopyalanabilir.

Dashboard store'un DB sync'i güncellenmeli

personalizedDashboardStore.ts içinde **saveToDB()** ve **loadFromDB()** doğrudan **/api/dashboard** çağırıyor. Yeni frontend'de bu URL'lerin **api-gateway**'e işaret etmesi gerekiyor.

18.4 Mevcut Veritabanı Şeması — Hangi Servis Yönetir?

Prisma Modeli	Yeni Sahibi	Açıklama
User	api-gateway	Kullanıcı kaydı ve profili
Account	api-gateway	OAuth provider bağlantısı
Session	api-gateway	JWT/session yönetimi
VerificationToken	api-gateway	Email doğrulama token'ı
UserActivityLog	api-gateway	Login/logout audit
ApiKey	exchange-service	Şifreli borsa API key'leri
Trade	exchange-service	İşlem geçmişi
Order	exchange-service	Emir geçmişi
PortfolioSnapshot	exchange-service	Portföy anlık görüntüsü
UserDashboard	api-gateway	Dashboard layout JSON
DashboardTemplate	api-gateway	Template kayıtları
UserFavorite	api-gateway	Favori modüller

18.5 Yeni Frontend'in Bağımlılık Haritası

Yeni frontend servisi kurulurken hangi şeylere ihtiyaç var:

YENİ FRONTEND

- api-gateway bağlantısı
 - Base URL (env: API_GATEWAY_URL)
 - Auth token mekanizması (Bearer JWT veya cookie)
 - CORS ayarları
- Direkt bağlantılar (api-gateway bypass)
 - wss://stream.binance.com (useTicker, useOrderBook)
 - wss://ws.okx.com (useTicker, useOrderBook)
 - wss://stream.bybit.com (useTicker, useOrderBook)
 - wss://ws-feed.exchange.coinbase.com (useOrderBook)
 - https://min-api.cryptocompare.com (useNews)
- Taşınabilir (değişmeden kopyalanır)
 - store/*.ts (Zustand store'ları)
 - hooks/useOrderBook.ts
 - hooks/useTicker.ts
 - hooks/useKlines.ts
 - hooks/useNews.ts
 - hooks/usePnL.ts
 - lib/personalized-dashboard/ (modül sistemi)
 - components/terminal/personalized-dashboard/ (modül bileşenleri)
- Güncellenmesi gerekenler
 - hooks/useExchangeKeys.ts (URL: /api/exchange/keys → api-gateway)
 - store/personalizedDashboardStore.ts (saveToDB/loadFromDB URL'leri)
 - components/auth/* (auth flow api-gateway'e göre)
 - tüm fetch("/api/...") çağrılarını → fetch(`\${API\_GATEWAY\_URL}/...`)

Özet

Tresaurio, modern web teknolojileriyle inşa edilmiş production-ready bir kripto trading terminalidir.

Kategori	Teknoloji
Framework	Next.js 15 (App Router)
Dil	TypeScript 5
State	Zustand v5
Auth	NextAuth.js v5 (JWT)
DB	PostgreSQL + Prisma
Styling	Tailwind CSS v4
Email	Resend
Şifreleme	AES-256-GCM

Kategori	Teknologi
WebSocket	Native WS + undici
Exchange	Binance, OKX, Bybit, Coinbase, Hyperliquid
Market Data	CoinGecko, CryptoCompare
Blockchain	Ethereum, BSC, Tron, Solana RPC
Deployment	Vercel / Docker / Self-hosted